

Apunte 02 — APIs y JSON: cómo se piden datos a un servidor

Proyecto Froth · aprender Machine Learning construyendo

1 La idea en 30 segundos

Tu PC (el *cliente*) le manda una petición por internet a otra computadora (el *servidor*) y esta responde con datos. Eso es todo. Una **API** (*Application Programming Interface*) es simplemente la ventanilla oficial que un servicio ofrece para que *programas* (no personas con navegador) le pidan cosas.

En Froth usamos la API de **OpenAlex**, un catálogo gratuito con ~250 millones de trabajos académicos.

Concepto: petición HTTP GET

Una petición se escribe como una URL con **parámetros** después del signo ?:

```
https://api.openalex.org/works?search=lepidolite flotation&per_page=25
```

Se lee: “del catálogo **works**, búscame **lepidolite flotation**, dame 25 por página”. En Python, la librería **requests** construye y envía esto por ti:

```
r = requests.get(url, params={...})
```

Concepto: JSON

El servidor responde en **JSON**: un formato de texto con la misma pinta que los diccionarios y listas de Python (`{"title": "...", "year": 2021}`). Por eso `r.json()` lo convierte directamente en objetos de Python con los que ya puedes trabajar. JSON es *el* idioma de intercambio de datos en internet.

2 Detalles de ciudadano educado

Las APIs públicas son un recurso compartido. Tres costumbres que nuestro `pull.py` ya practica:

1. **Identifícate** (*polite pool*): mandamos nuestro email en el parámetro `mailto`. OpenAlex da mejor servicio a quien da la cara.
2. **No bombardees**: entre búsqueda y búsqueda esperamos 0.3 s (`time.sleep(0.3)`). Miles de peticiones por segundo tumban servicios.
3. **Pide solo lo que necesitas**: el parámetro `select` lista los campos que queremos (título, año, autores...). Menos datos = más rápido para todos.

Concepto: el truco del abstract invertido

OpenAlex no guarda el resumen como texto corrido sino como un *índice invertido*: un diccionario `{palabra: [posiciones donde aparece]}`. Es una curiosidad real de esta API (lo hacen por temas legales y de eficiencia). Nuestra función `_reconstruct_abstract()` lo vuelve a montar: coloca cada palabra en su posición y las une. Primer ejemplo del proyecto de que **los datos reales vienen “sucios”** y hay que trabajarlos.

Pruébalo tú (teclado en mano)

Pega esta URL en tu navegador (¡una API también responde ahí!) y mira el JSON crudo:
`https://api.openalex.org/works?search=lepidolite%20flotation&per_page=2`
Busca los campos `title`, `publication_year` y `abstract_inverted_index` en la respuesta.

En una frase

Una API es una ventanilla para programas: mandas una URL con parámetros, te responden JSON, y `requests + r.json()` lo convierten en diccionarios de Python.